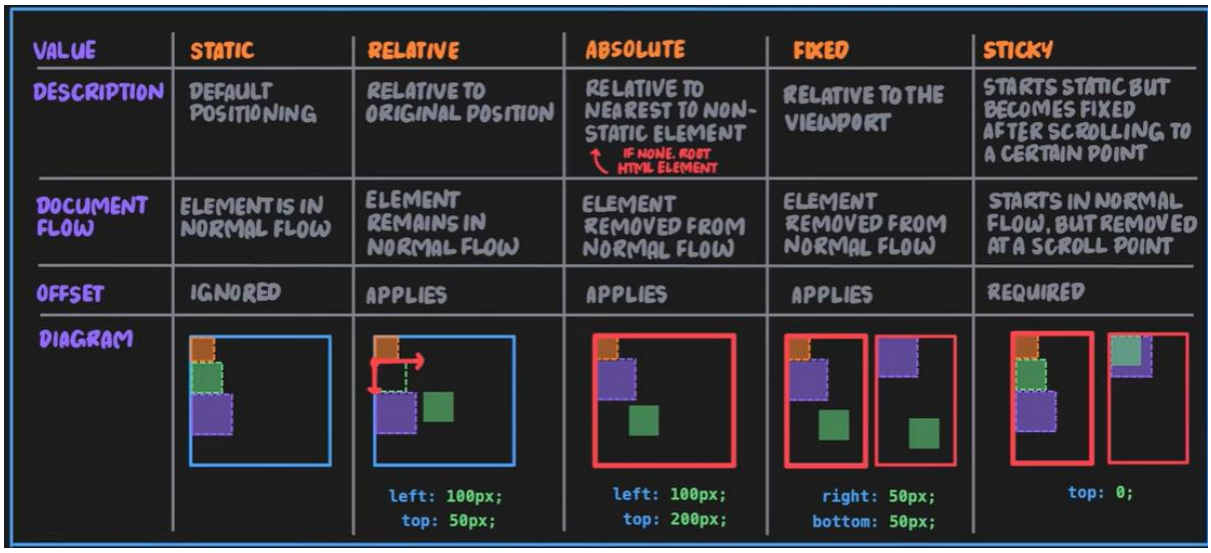
Date : 26-03-2026

Positioning Contexts:



An **absolutely positioned element** is positioned relative to its **nearest positioned ancestor**.

- If **no parent/ancestor** has a position value other than static:
    - Default is position: static for all elements.
    - The element is positioned relative to the **initial containing block**.

Initial Containing block it is basically a view port (browser window) same size as the screen.

Even if the element is inside <body> in HTML:

- It **ignores normal flow**
- Positions itself relative to the **viewport**

Z-index:

z-index controls which element should appear on the top when elements overlap.

Without z-index the default behaviour is

1) If multiple elements overlap - **Positioned elements** (absolute, relative, etc.) appear **above non-positioned elements**
2) If both are positioned - The element **later in HTML (source order)** appears on top

Ex:

z-index: 1;

takes only numbers and positive numbers moves forward negative numbers moves backwards.

Fixed Positioning:

This works in exactly the same way as absolute positioning, with one key difference: whereas absolute positioning fixes an element in place relative to its nearest positioned ancestor (the initial containing block if there isn't one), **fixed positioning** fixes an element in place relative to the visible portion of the

viewport. This means that you can create useful UI items that are fixed in place, like persistent navigation menus that are always visible no matter how much the page scrolls.

Sticky Positioning:

There is another position value available called position: sticky, which is somewhat newer than the others. This is basically a hybrid between relative and fixed position. It allows a positioned element to act like it's relatively positioned until it's scrolled to a certain threshold (e.g., 10px from the top of the viewport), after which it becomes fixed

```
.positioned {
  position: sticky;
  top: 30px;
  left: 30px;
}
```

Scrolling index:

An interesting and common use of position: sticky is to create a scrolling index page where different headings stick to the top of the page as they reach it.

Stacking Context:

A stacking context is like a separate layer/group of elements on a webpage.

It works on the z-axis (depth) → controls which element appears on top or below.

Think of it like:
A box of elements stacked together independently

Elements inside a stacking context:

- Stack only with each other
- Do NOT affect elements outside
- Each stacking context acts as a single unit in its parent
- `z-index` works only within the same stacking context
- Stacking contexts can be nested (parent → child hierarchy)
- Only descendants are considered for stacking inside a context

When Stacking context is created:

Position-based

- position: absolute/relative + z-index ≠ auto
- position: fixed
- position: sticky

Layout based

- Flex items with z-index
- Grid items with z-index

Stacking contexts can be contained in other stacking contexts, and they can together create a hierarchy of stacking contexts.

Modern Layout:

CSS Flexbox:

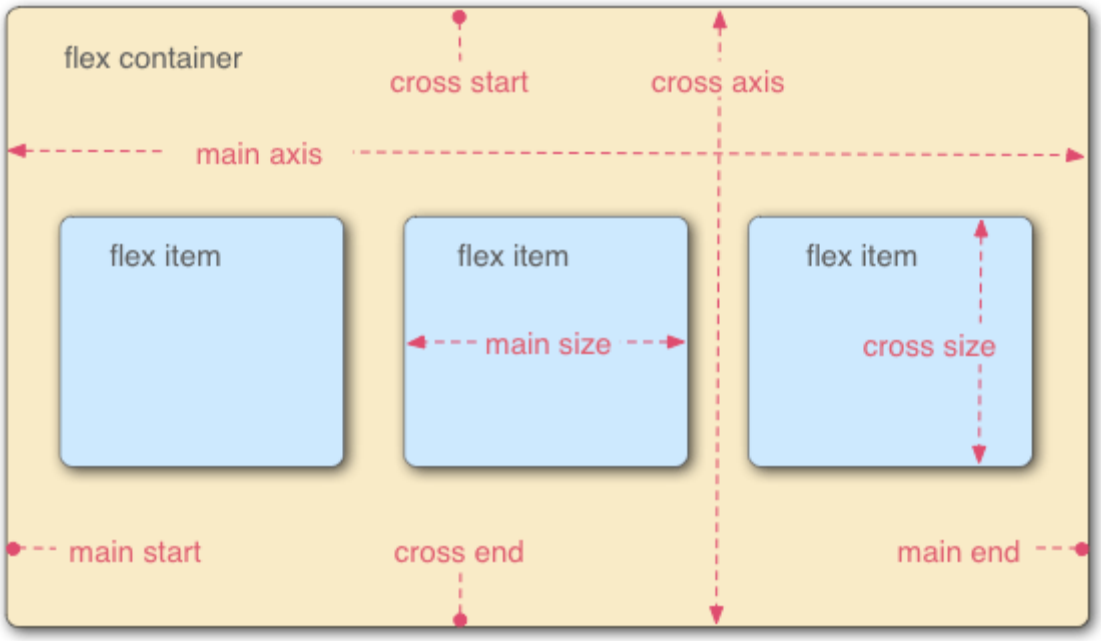
It is a short for flexible short layout. This layout model simplifies creating complex layouts.

The flexbox model is bidirectional – that means we can arrange our elements in either row or column or both.

To start with here we use display: flex or inline-flex.

The flex model:

When elements are laid out as flex items, they are laid out along two axes:



- The **main axis** is the axis running in the direction the flex items are laid out in (for example, as a row across the page, or a column down the page.) The start and end of this axis are called the **main start** and **main end**. The length from the main-start edge to the main-end edge is the **main size**.
- The **cross axis** is the axis running perpendicular to the direction the flex items are laid out in. The start and end of this axis are called the **cross start** and **cross end**. The length from the cross-start edge to the cross-end edge is the **cross size**.
- The parent element that has display: flex set on it is called the **flex container**.
- The items laid out as flexible boxes inside the flex container are called **flex items.**

Columns or rows:

Flexbox provides a property called flex-direction that specifies which direction the main axis runs (which direction the flexbox children are laid out in). By default this is set to row, which causes them

to be laid out in a row in the direction your browser's default language works in (left to right, in the case of an English browser).

flex-direction: column;

Wrapping:

One issue that arises when you have a fixed width or height in your layout is that eventually your flexbox children will overflow their container, breaking the layout.

When we give the height and width that may go beyond the page by default, the browser tries to place all the flex items in a single row if the flex-direction is set to row or a single column if the flex-direction is set to column.

```
section {

  flex-wrap: wrap;

}
```

This will create the multiple rows so make the page looks without overflow.

flex-direction property value to row-reverse then we'll see that you still have your multiple row layout, but it starts from the opposite corner of the browser window and flows in reverse.

Flex-flow shorthand:

We can replace this ,

flex-direction: row;

flex-wrap: wrap;

with this,

flex-flow: row wrap;

Flexible sizing of flex items:

Using

```
article {

  flex: 1;

}
```

If we give the flex:1 then the available size of the screen will be splits into equal parts based on what our need we can give the value and conditions based on that.

Flex: shorthand vs Longhand properties

flex is a shorthand property that can specify up to three different values:

1. flex-grow

- Defines how much a flex item can grow
- Takes a unitless value (like 1, 2, 3…)

- Works when there is extra space in container

Example:

  - flex-grow: 1 → takes equal space
  - flex-grow: 2 → takes twice the space of others

2. flex-shrink

- Defines how much a flex item can shrink
- Works when items are overflowing container
- Also takes unitless value

Example:

  - flex-shrink: 1 → normal shrinking
  - flex-shrink: 0 → item won't shrink

3. flex-basis

- Defines the initial size of a flex item
- Can use units like: px, %, auto

Example:

  - flex-basis: 100px → starts with 100px width

Horizontal and Vertical Alignment:

align-items (Cross Axis)

- Controls vertical alignment (in row layout)
- Common values:
  - center → center items
  - flex-start → top
  - flex-end → bottom
- Default: stretch

3. justify-content (Main Axis)

- Controls horizontal alignment
- Common values:
  - center → middle
  - space-around → equal space around items
  - space-between → space only between items
  - flex-start / flex-end

4. align-self (Individual Item)

- Overrides align-items for one item
- Example:

```
align-self: flex-end;
```

- Used for custom alignment of a single element

The justify-items property is ignored in flexbox layouts.

Ordering flex items:

Flexbox also has a feature for changing the layout order of flex items without affecting the source order. This is another thing that is impossible to do with traditional layout methods.

```
button:first-child {
  order: 1;
}
```

- By default, all flex items have an order value of 0.
- Flex items with higher specified order values will appear later in the display order than items with lower order values.
- Flex items with the same order value will appear in their source order. So if you have four items whose order values have been set as 2, 1, 1, and 0 respectively, their display order would be 4th, 2nd, 3rd, then 1st.
- The 3rd item appears after the 2nd because it has the same order value and is after it in the source order.

We can set negative order values to make items appear earlier than items whose value is 0.

Nested flex boxes:

It's possible to create some pretty complex layouts with flexbox. It's perfectly OK to set a flex item to also be a flex container, so that its children are also laid out like flexible boxes.

Grid Layouts:

A grid is a collection of horizontal and vertical lines creating a pattern against which we can line up our design elements. They help us to create layouts in which our elements won't jump around or change width as we move from page to page, providing greater consistency on our websites.

A grid will typically have **columns**, **rows**, and then gaps between each row and column. The gaps are commonly referred to as **gutters**.

Creating a grid:

Using,

```
.container {
  display: grid;
```

```
  grid-template-columns: 200px 200px 200px;
}
```

This will create a grid to make the website responsive the grid is useful

Flexible grids with fr units:

creating grids using lengths and percentages, we can use fr.

fr - fr unit represents one fraction of the available space in the grid container to flexibly size grid rows and columns.

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

The fr unit distributes *available* space, not *all* space. Therefore, if one of your tracks has something large inside it, there will be less free space to share.

Gaps between tracks:

To create gaps between tracks, we use the properties:

- column-gap for gaps between columns
- row-gap for gaps between rows
- gap as a shorthand for both

These gaps can be any length unit or percentage, but not an fr unit.

Repeating track listings

You can repeat all or merely a section of your track listing using the CSS repeat() function.

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

You'll now get three 1fr tracks just as before. The first value passed to the repeat() function specifies the number of times you want the listing to repeat, while the second value is a track listing, which may be one or more tracks that you want to repeat.

Implicit and explicit grids:

- **Explicit grid** is created using grid-template-columns or grid-template-rows.
- **Implicit grid** extends the defined explicit grid when content is placed outside of that grid, such as into the rows by drawing additional grid lines.

- When you place items **outside the defined grid**, CSS automatically creates **extra rows/columns** these are called **implicit grid tracks.**

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
  gap: 20px;
}
```

By default:

grid-auto-rows: auto;

- Rows/columns will **adjust size based on content**
- Big content → bigger row
- Small content → smaller row

We can fix the size using:

grid-auto-rows: 100px;

Now:

- All automatically created rows will be **exactly 100px height**

For columns:

- `grid-auto-columns: 100px;`

Controls width of **implicit columns**

Minmax() function:

The minmax() function lets us set a minimum and maximum size for a track, for example, minmax(100px, auto). The minimum size is 100 pixels, but the maximum is auto, which will expand to accommodate more content.

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(50px, auto);
  gap: 20px;
}
```

Using grid-template-columns: repeat(auto-fit, minmax(230px, 1fr)), CSS Grid automatically creates as many columns as can fit in the container. Each column has a minimum width of 230px and can expand up to 1fr to share remaining space evenly. This makes the layout responsive by adjusting the number and size of columns based on available space.

Line based placement:

We now move on from creating a grid to placing things on the grid. Our grid always has lines — these are numbered beginning with 1 and relate to the writing mode of the document. For example, column line 1 in English (written left-to-right) would be on the left-hand side of the grid and row line 1 at the top, while in Arabic (written right-to-left), column line 1 would be on the right-hand side.

To position items along these lines, we can specify the start and end lines of the grid area where an item should be placed. There are four properties we can use to do this:

- grid-column-start
- grid-column-end
- grid-row-start
- grid-row-end

These properties accept line numbers as their values, so we can specify that an item should start on line 1 and end on line 3, for example. Alternatively, you can also use shorthand properties that let you specify the start and end lines simultaneously, separated by a forward slash /:

- grid-column shorthand for grid-column-start and grid-column-end
- grid-row shorthand for grid-row-start and grid-row-end

Positioning with grid template areas:

An alternative way to arrange items on your grid is to use the grid-template-areas property and give the various elements of your design a name.

```
.container {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar content"
    "footer footer";
  grid-template-columns: 1fr 3fr;
  gap: 20px;
}
header {
  grid-area: header;
}
main {
  grid-area: content;
```

```
}

aside {

  grid-area: sidebar;

}

footer {

  grid-area: footer;

}
```

Nesting grids and subgrid:

It's possible to nest a grid within another grid, creating a "subgrid". You can do this by setting the display: grid property on an item in the parent grid.

We can use subgrid on grid-template-rows and grid-template-columns properties. This allows you to leverage the tracks defined in the parent grid.

```
.container {

  display: grid;

  grid-template-columns: repeat(4, 1fr);

  grid-template-rows: repeat(1, 1fr);

  gap: 10px;

}

.subgrid {

  grid-column: 1 / 4;

  grid-row: 2 / 4;

  display: grid;

  gap: inherit;

  grid-template-columns: subgrid;

  grid-template-rows: 2fr 1fr;

}
```

Grid Frameworks:

Numerous grid frameworks are available — these are prebuilt CSS systems that provide features such as 12- or 16-column grids, utility classes for spacing and alignment, and responsive design via breakpoints.

Responsive design:

Media queries:

Media queries allow us to run a series of tests (for example, whether the user's screen is greater than a certain width or resolution) and apply CSS selectively to style the page appropriately for the user's needs.

For example, the following media query tests to see if the current web page is being displayed as screen media (therefore not a printed document) and the viewport is at least 80rem wide.
The .container rule will only be applied if these two things are true.

```
@media screen and (width >= 80rem) {
  .container {
    margin: 1em 2em;
  }
}
```

A common approach when using media queries is to create a simple single-column layout for narrow-screen devices (for example, mobile phones), then check for wider screens and implement a multiple-column layout when you know that you have enough screen width to handle it. Designing for mobile first is known as **mobile first** design.

Responsible layout techniques:

Responsive sites are built on flexible grids, meaning you don't need to target every possible device size with pixel perfect layouts.

Several layout methods — including Flexbox, and CSS Grid — are responsive by default.

Flexbox:

In flexbox, flex items shrink or grow, distributing space between the items according to the space in their container. By changing the values for flex-grow and flex-shrink you can indicate how you want the items to behave when they encounter more or less space around them.

CSS grid:

In CSS grid layout the fr unit allows the distribution of available space across grid tracks. The next example creates a grid container with three tracks sized at 1fr. This will create three column tracks, each taking one part of the available space in the container.

Responsive images and media:

To ensure media is never larger than its responsive container, the following approach can be used:

```
img,
picture,
video {
  max-width: 100%;
}
```

- Always make sure to use an appropriate image format for your website images (such as PNG or JPG), and make sure to optimize the file size using a graphics editor before you put them on your website.
- You can make use of CSS features like gradients and shadows to implement visual effects without using images.
- You can use media queries inside the media attribute on <source> elements nested inside <video>/<audio> elements to serve video/audio files as appropriate for different devices (responsive video/audio).

Responsive typography:

Responsive typography describes changing font sizes within media queries or using viewport units to reflect lesser or greater amounts of screen real estate.

```
html {
  font-size: 1em;
}

h1 {
  font-size: 2rem;
}

@media (width >= 1200px) {
  h1 {
    font-size: 4rem;
  }
}
```

In this example, we want to set our level 1 heading to be 4rem, meaning it will be four times our base font size. That's a really large heading! We only want this jumbo heading on larger screen sizes, therefore we first give the heading a smaller size of 2rem, then use media queries to overwrite it with the larger size if we know that the user has a screen width of at least 1200px.

Using viewport:

Viewport units vw can also be used to enable responsive typography, without the need for setting breakpoints with media queries. 1vw is equal to one percent of the viewport width, meaning that if you set your font size using vw, it will always relate to the size of the viewport.

```
h1 {
  font-size: 6vw;
}
```

The problem with doing the above is that the user loses the ability to zoom any text set using the vw unit, as that text is always related to the size of the viewport. **Therefore you should never set text using viewport units alone**.

There is a solution, and it involves using calc(). If you add the vw unit to a value set using a fixed size such as ems or rems then the text will still be zoomable. Essentially, the vw unit adds on top of that zoomed value:

```
h1 {
  font-size: calc(1.5rem + 4vw);
}
```

The viewport meta tag:

The viewport meta tag (<meta name="viewport" content="width=device-width, initial-scale=1">) is used to make a webpage properly fit and display on different screen sizes, especially mobile devices. It tells the browser to match the screen's width and display the content at the correct scale instead of shrinking it.

Mobile browsers originally used a default viewport width of around 980px to display desktop websites, which caused pages to appear zoomed out on smaller screens. Users then had to manually zoom and scroll to read content, leading to a poor user experience.

By setting width=device-width, the browser uses the actual width of the device, ensuring that responsive designs work as intended. This allows media queries and breakpoints to correctly adjust layouts for different screen sizes.

Therefore, this meta tag is essential for modern web development, as it ensures proper responsiveness and usability across mobile devices and should always be included in the <head> section of a webpage.